Arrays und Referenzen

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Grundlagen von Arrays
- Eindimensionale Arrays
- Umgang mit Arrays

Was wird in diesem Foliensatz besprochen?

- Referenzen in Java
- Speicherverwaltung mit Stack und Heap

Was lernen Sie kennen?

- Arrays und Referenzen
- Unterschied zwischen Stack und Heap
- Umgang mit Array-Referenzen

Zuweisung von Arrays

Array-Variable ist eine **Referenz** auf das eigentliche Array

Einer Array-Variable vom Typ *x* kann immer nur ein Array vom Typ *x* zugewiesen werden

- Länge kann aber variieren

Es wird nur die Adresse kopiert, nicht der Inhalt

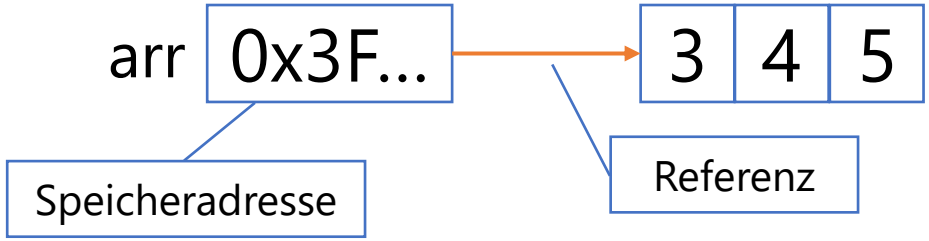
Elementare Typen – Referenztypen

int-Variable (schematisch)

```
int x = 1;
```

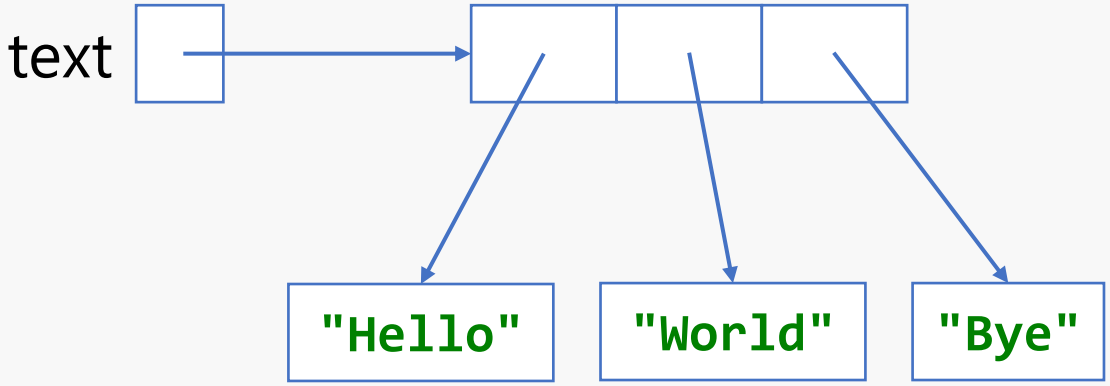
int-Array (schematisch)

```
int[] arr = {3, 4, 5};
```



Arrays von Strings (schematisch)

```
String[] text = new String[]{ "Hello", "World", "Bye" };
```



Zuweisung bei Array-Variablen (1)

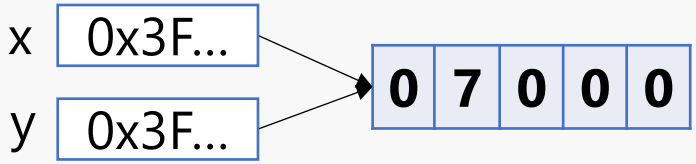
```
int[] x = new int[10], y;
y = x;
y[1] = 7;
System.out.println(y[0] + " " + x[0]);
System.out.println(y[1] + " " + x[1]);
```

```
0 0
7 7
```

Warum hat x[1] auch den Wert 1?

Zuweisung bei Array-Variablen (2)

```java
int[] x = new int[10], y;
y = x;
y[1] = 7;
System.out.println(y[0] + " " + x[0]);
System.out.println(y[1] + " " + x[1]);
```



x und y sind Array-Variablen

x und y zeigen auf den gleichen Speicherbereich

- x und y sind Aliase

Die Zuweisung bei y[1] verändert auch x[1]

Stack-Speicher (Wiederholung)

Methoden-Frames mit lokalen Variablen, Zwischenergebnissen etc.

Spielt bei **Methodenaufrufen** und beim **Zurückkehren** aus Methoden eine Rolle

StackOverflowError, falls zu wenig Stack-Speicher vorhanden (siehe Rekursion)

Heap-Speicher

Objekte und Arrays

Wird beim **Start der JVM** angelegt

Nicht mehr benutzter Speicher wird **automatisch eingesammelt** (siehe späteres Beispiel)

OutOfMemoryError, falls zu wenig Heap-Speicher vorhanden

Freigabe des Array-Speichers

Erfolgt durch das System	Garbage-Collector
• Speicher muss nicht explizit freigegeben werden	• Nicht benutzter Speicher wird zur Laufzeit automatisch vom Garbage Collector eingesammelt

Automatische Freigabe

Voraussetzung für das Einsammeln

- **Array wird nicht mehr referenziert**

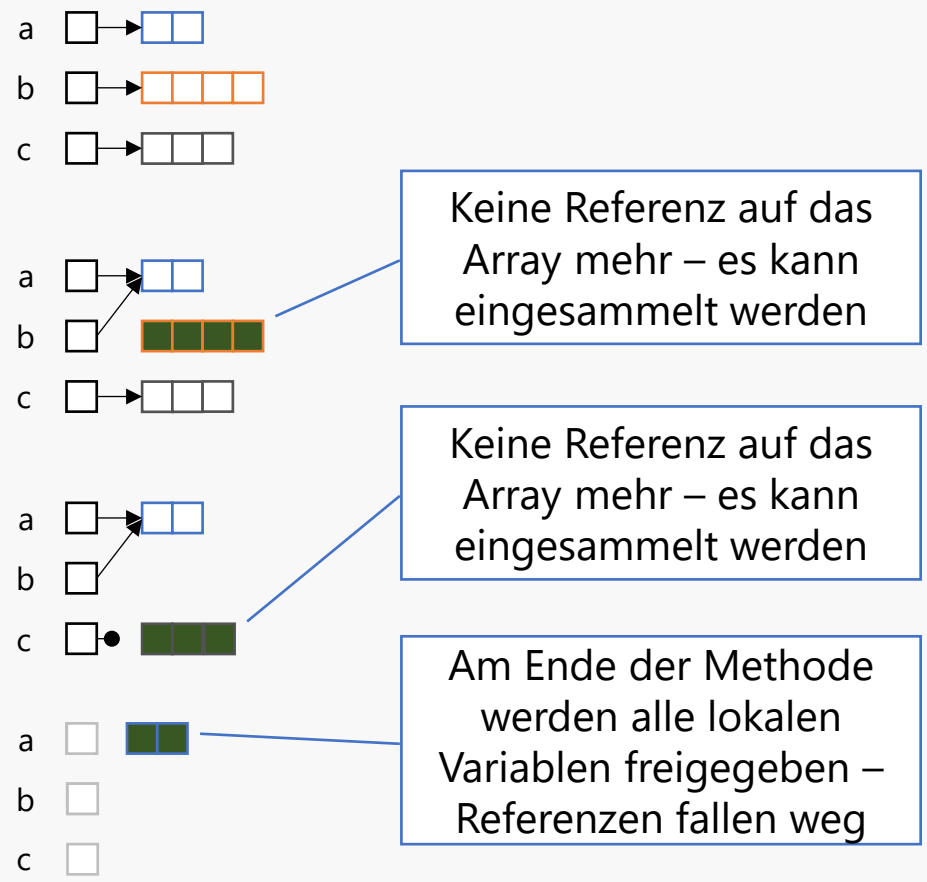
Erreichbarkeit unterbinden

- Einer Array-Variable ein anderes Array zuweisen
- Einer Array-Variable den Wert null zuweisen

```
x = null;
```

Beispiel

```
private static void test() {
    int[] a = new int[2];
    int[] b = new int[4];
    int[] c = new int[3];
    …
    …
    …
    b = a;
    …
    …
    …
    c = null;
    …
    …
}
```



Kopieren von Arrays – eigene Lösung

```
int[] original = {3, 4, 5, 6, 7};
int[] copy = new int[original.length];
for(int i = 0; i < original.length; i++) {
    copy[i] = original[i];
}
copy[0] = 8;
System.out.println(original[0]);
```

Elementweise kopieren

Hier wird 3 ausgegeben

Kopieren von Arrays – Java API

Methode System.arraycopy
• Effiziente Methode • Kopieren von Abschnitten bzw. ganzen Arrays • API genau lesen!

Beispiel (System.arraycopy verwenden)

```
public class ArrayCopy {

    public static void main(String[] args) {
        int[] original = {3, 4, 5, 6, 7};
        int[] copy = new int[original.length];
        System.arraycopy(original, 0, copy, 0, original.length);
        for (int j : copy) {
            System.out.print(j + " ");
        }
        System.out.println();
        System.arraycopy(original, 0, original, 2, 3);
        for (int j : original) {
            System.out.print(j + " ");
        }
    }

}
```

```
3 4 5 6 7
3 4 3 4 5
```

Elementweise kopieren

Abschnitt (die ersten 3 Elemente) ab Position 2 in das Array selbst kopieren

Klonen von Arrays

```java
public class Test {
    public static void main(String[] args) {
        int[] original = {3, 4, 5, 6, 7};
        int[] copy = original.clone();
        for (int k : copy) {
            System.out.print(k + " ");
        }
        System.out.println();
        original[1] = 10;
        for (int j : original) {
            System.out.print(j + " ");
        }
        System.out.println();
        for (int j : copy) {
            System.out.print(j + " ");
        }
    }
}
```

Auf jedes Array kann die **clone-Methode** aufgerufen werden und erzeugt eine Kopie von dem Array

```
3 4 5 6 7
3 10 5 6 7
3 4 5 6 7
```

Vergleiche bei Arrays

Operator == (auch !=) bei Arrays meist nicht geeignet

- == vergleicht den Inhalt der Speicherzellen
- Vergleicht daher nur, ob Arrays **die gleichen Referenzen** haben
- Vergleicht **nicht den Inhalt** der Arrays

Inhalt zweier eindimensionaler Arrays vergleichen

```
int[] original = …;
int[] copy = …;
…
boolean eq = (original.length == copy.length);
for (int i = 0; eq && i < original.length; i++) {
    eq = (original[i] == copy[i]);
}
System.out.println(eq ? "equal" : "not equal");
```

Hinweis: In der Praxis gibt es dafür Bibliotheksmethoden (siehe Foliensatz über Hilfsklasse Arrays)!

Gleichheit bei Strings

Gleichheit
• Zwei Zeichenketten sind gleich, wenn sie die gleichen Längen haben und die gleichen Zeichen in gleicher Reihenfolge enthalten • Methode `equals` für Vergleich benutzen

Gleichheit bei Strings (Beispiel)

```
System.out.println("ab".equals("ab"));  // true
System.out.println("ab".equals("ba"));  // false
String s = "ab", t = "cd";
String u = "ab", v = t;
System.out.println(s.equals(u));       // true
System.out.println(s == u);            // true
System.out.println(t.equals(v));       // true
System.out.println(t == v);            // true
System.out.println((s+t).equals(u+v)); // true
System.out.println((s+t) == (u+v));    // false
```

Unterschied zu Arrays!
Compiler optimiert, d. h. das String-Literal „ab“ gibt es nur einmal!

Zur Laufzeit keine Optimierung!